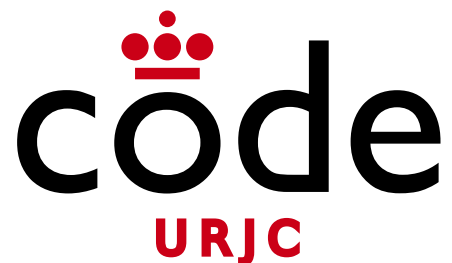


Grado en Ingeniería de la Ciberseguridad

Desarrollo Web Seguro

Bloque III: Vulnerabilidades, contramedidas

Tema 2: Fortificación de servidores



©2026

Sergio Pérez Peló, Raúl Martín Santamaría

Algunos derechos reservados

Este documento se distribuye bajo la licencia
"Atribución-CompartirIgual 4.0 Internacional"
de Creative Commons Disponible en

<https://creativecommons.org/licenses/by-sa/4.0/deed.es>

Contenidos

- Vamos a ver:
 - Recomendaciones generales de seguridad
 - Cómo ponerlo en práctica en el contexto de aplicaciones web Spring Boot, como la que habéis desarrollado.

Contexto

- Tenemos una versión inicial de nuestra aplicación. Queremos desplegarla y hacerla pública.
- ¿Estamos seguros de que nuestra aplicación no tiene fallos de seguridad? **¿Podemos garantizarlo?**
- ✉ Asumimos que nos van a atacar, queremos **dificultar** la labor del atacante.

Estrategia

1. Mínimo Privilegio
2. Mínima Exposición
3. Defensa en profundidad
/ Zero Trust

Ninguna de estas medidas va a hacernos invulnerables, pero van a imponer un coste mayor a los atacantes.

El atacante desde el punto de vista de la empresa:



Atacante promedio:



Mínimo privilegio

- **Crea un usuario / rol específico para cada app.**
Y asígnale los privilegios estrictamente necesarios.
- **Ej:** si ejecutas tu app desde tu usuario:
 - Tiene acceso a todos tus ficheros.
 - Si alguien consigue RCE y eres admin / sudoer, puede ser trivial escalar privilegios.

Mínimo privilegio

- **Caso práctico 1: quiero que mi app escuche en puerto 80 / 443.**
 - Como son puertos reservados, un usuario normal no puede lanzar la app en ellos.
 - Posible solución: **sudo** java -jar ... 
 - Alternativas:
 - Proxy para redirigir tráfico. Ej: NGINX
 - CAP_NET_BIND_SERVICE
 - `iptables -t nat -A OUTPUT -o lo -p tcp --dport=80 -j REDIRECT --to-port=8080`

Mínimo privilegio

- **Caso práctico 2: BBDD**

- ¿Necesitamos realmente que Spring tenga permiso de *root* en la BBDD?

A) BBDD instalada
previamente:

```
CREATE USER 'user'@'localhost'  
IDENTIFIED BY userpw';
```

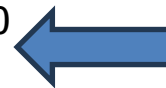
```
CREATE DATABASE myapp;
```

```
GRANT ALL PRIVILEGES ON myapp.* TO  
'user'@'localhost';
```

B) Si usamos Docker:
environment:

```
MYSQL_ROOT_PASSWORD: secret  
MYSQL_DATABASE: miapp  
MYSQL_USER: user  
MYSQL_PASSWORD: userpw
```

Restringir más si es
posible. Ej: solo lectura
de algunas tablas.



Mínima exposición

- Minimiza apps, servicios y librerías instaladas
 - Ejemplo: usa contenedores pequeños, solo lo que necesites.
- Minimiza los servicios expuestos
 - Solo debería ser accesible desde Internet el servidor web, nunca la BBDD u otras dependencias.
¿Y si necesito acceso directo?  VPN / *Port forwarding*
 - Si usas una CDN (Ej: Cloudflare), podemos incluso bloquear el acceso a todos excepto las IPs de la CDN.

Defense in Depth / Zero Trust

- Asume que cualquier parte de la app podría estar comprometida.
- Aprovecha las medidas de seguridad de tu SO
 - Ej. Linux: cgroups, SECCOMP, AppArmor, SELinux...
- Mitigaciones a nivel de componentes:
 - Valida los datos en cada “nivel” : BBDD, Back, Front.
- Log de comportamientos sospechosos
 - ✉ Facilita investigaciones posteriores.

Defense in Depth / Zero Trust

- **Caso práctico:** validar y reportar accesos raros a disco (métodos auxiliares a nivel de servicio)

```
Path requestedPath = BASE_DIRECTORY.resolve(userInputPath).normalize();

if (!requestedPath.startsWith(BASE_DIRECTORY)) {
    logger.warning("Path traversal detected: " + userInputPath);
    return;
}

File file = requestedPath.toFile();
if (!file.exists() || !file.isFile()) {
    logger.warning("Invalid file: " + requestedPath);
    return;
}
file.read ....
```

Conclusiones

Hay que trabajar para ser
**OBJETIVOS
DUROS**



Si los malos entran con un 0-day, es que estás haciendo las cosas bien.

Fuente imagen, charla recomendada:

11 años luchando contra APT: ¿Qué hemos aprendido? - Antonio Sanz

https://www.youtube.com/watch?v=vK7a_LtIY-Q